

Design of a Use Case Diagram for a Software System & Defining User Stories

Course: Software Development Project I

Lab: Use Case Diagram & User Stories

Project: Ghuri-Phiri – A Smart Tourist Guide & Budget Planner

Group: 5 Members

Punam (23524202064)

Rifat (23524202096)

Mehrab (23524202112)

Orce (23524202136)

1. Objectives

- I want to learn about use case diagrams in UML.
 - Use case diagrams in UML are really important.
 - I need to figure out how to find the actors and use cases of a system.
 - Then I can use use case diagrams, in UML to show what a system can do in a picture.
-

2. Problem Analysis

Draw a Use Case Diagram for the Ghuri-Phiri system — a web-based smart tourist guide and budget planner for Bangladesh.

The system serves three types of users: general travelers who browse destinations and plan budgets, researchers/students who seek detailed cultural and historical information, and

administrators who manage destination data and pricing. The system integrates with external APIs (Google Maps, OpenWeatherMap) and an internal MySQL database.

The key interactions to be captured are:

- Travelers browsing and filtering tourist destinations.
 - Travelers viewing place details including cost estimates and weather.
 - Travelers accessing map navigation.
 - Administrators logging in and updating pricing data.
 - The system communicating with external APIs for weather and map services.
-

3. Use Case Diagram

A Use Case Diagram is a type of UML (Unified Modeling Language) diagram that represents the functional requirements of a system. It shows the interaction between actors (users or external systems) and the system.

A Use Case Diagram is a behavioral UML diagram that models system functionality using actors and use cases. It illustrates the interaction between users and the system.

3.1 Key Components

Actor:

- Represents a user or external entity (e.g., Traveler, Admin, Google Maps API).
- Represented by stick figures; they denote users or external entities interacting with the system.

Use Case:

- Represents a function or service provided by the system.
- Represented by ovals; they denote system functionalities.

System Boundary:

- Defines the scope of the system.
- A rectangle that defines the scope of the system.
- All use cases within the boundary belong to the system.

Packages:

- Used to group related elements, represented like folders.

Relationships: Lines connecting actors and use cases, showing interaction.

- **Association** — interaction between actor and use case (solid line between actor and use case).
 - **Generalization** — solid arrow indicating inheritance relationship.
 - **Include** — mandatory inclusion of one use case into another.
 - **Extend** — optional extension of a base use case.
-

4. Methodology

- Identify the system scope.
- Identify all actors interacting with the system.
- Identify all possible use cases.
- Establish relationships between actors and use cases.
- Draw the diagram with proper notation.

Step 1: Identify System Scope

The system scope is the Ghuri-Phiri web application — covering destination discovery, budget calculation, live weather display, map navigation, and admin data management. External API services (OpenWeatherMap, Google Maps) fall outside the system boundary but interact with it.

Step 2: Identify All Actors

Actor	Type	Description
Traveler	Primary Actor ▾	Browses destinations, filters by category/budget, views details, checks costs and weather, accesses maps
Researcher / Student	Primary Actor ▾	Views detailed historical and cultural information about heritage destinations
Administrator	Primary Actor ▾	Logs into secure dashboard, updates transport fares and hotel rates
OpenWeatherMap API	External System ▾	Provides real-time weather data to the system
Google Maps API	External System ▾	Provides map view and routing functionality

Step 3: Identify All Use Cases

Traveler Use Cases:

- Browse Destinations by Division
- Filter Destinations (by Category / Budget)
- View Place Details
- Calculate Trip Budget
- View Live Weather
- View Location on Map
- Get Route to Destination

Researcher / Student Use Cases:

- Browse Destinations by Division
- View Place Details
- View Historical & Cultural Information

Administrator Use Cases:

- Login to Admin Dashboard
- Update Transport Fare
- Update Hotel Rate
- Add New Destination
- Manage Destination Records

System-Initiated Use Cases (via external APIs):

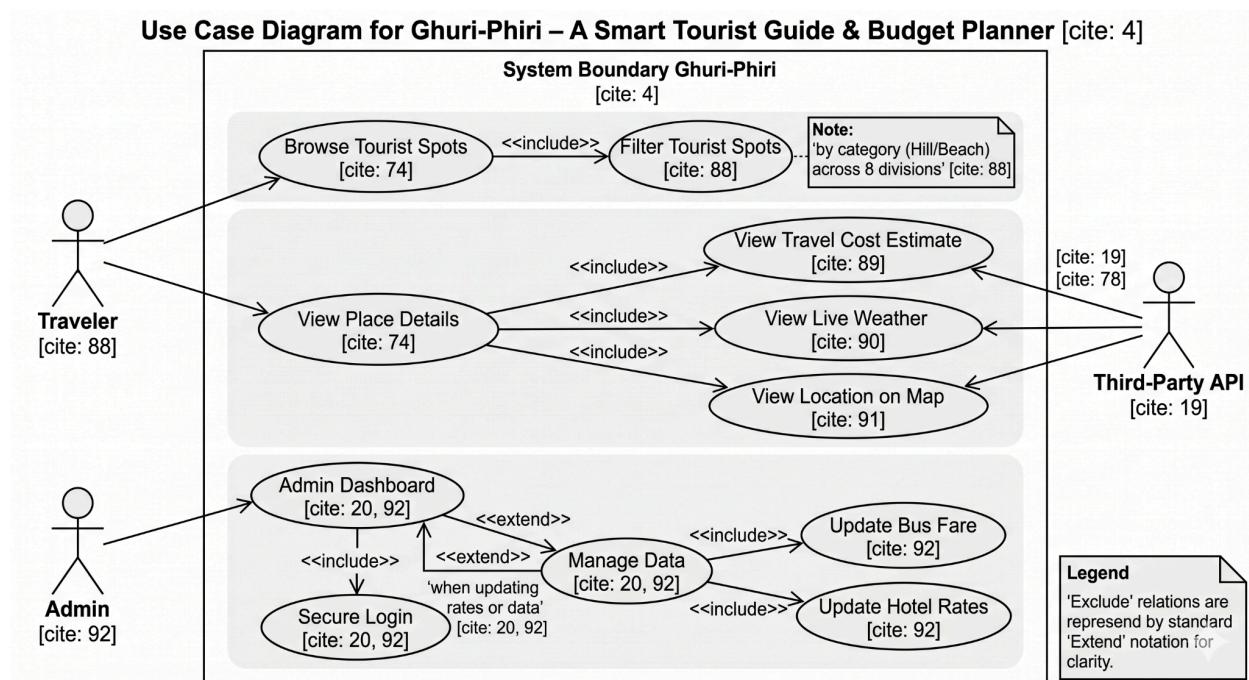
- Fetch Weather Data (OpenWeatherMap API)
- Fetch Map & Route Data (Google Maps API)

Step 4: Establish RelationshipsStep

Relationship	Type	Description
Traveler → Browse Destinations by Division	Association ▾	Direct interaction ▾
Traveler → Filter Destinations	Association ▾	Direct interaction ▾
Traveler → View Place Details	Association ▾	Direct interaction ▾
View Place Details → Calculate Trip Budget	Include ▾	Budget is always show... ▾

View Place Details → View Live Weather	Include ▾	Weather badge is alwa... ▾
View Place Details → View Location on Map	Include ▾	Map link is always pres... ▾
View Location on Map → Get Route to Destination	Extend ▾	Route is optional — us... ▾
Researcher/Student → Browse Destinations by Division	Association ▾	Shared with Traveler ▾
Researcher/Student → View Place Details	Association ▾	Shared with Traveler ▾
Researcher/Student → View Historical & Cultural Information	Association ▾	Additional detail availab... ▾
Traveler → Researcher/Student	Generalization ▾	Researcher is a special... ▾
Administrator → Login to Admin Dashboard	Association ▾	Mandatory first step ▾
Login to Admin Dashboard → Update Transport Fare	Include ▾	Admin must be logged i... ▾
Login to Admin Dashboard → Update Hotel Rate	Include ▾	Admin must be logged i... ▾
Login to Admin Dashboard → Add New Destination	Include ▾	Admin must be logged i... ▾
View Live Weather → Fetch Weather Data	Include ▾	System must call Open... ▾
View Location on Map → Fetch Map & Route Data	Include ▾	System must call Googl... ▾
OpenWeatherMap API → Fetch Weather Data	Association ▾	External system interac... ▾
Google Maps API → Fetch Map & Route Data	Association ▾	External system interac... ▾

Step 5: Use Case Diagram



5. User Stories

A **User Story** is a short, simple description of a system requirement written from the user's perspective. It focuses on what the user needs and why. User stories are widely used in Agile

and Incremental development to capture functional requirements in a clear and understandable way.

Format: As a [user], I want to [action] so that [benefit].

Traveler User Stories:

- As a traveler, I want to browse tourist spots by division so that I can explore all destinations available in a specific region of Bangladesh.
- As a traveler, I want to filter destinations by category (Hill/Beach/Heritage) across all 8 divisions so that I can find places that match my personal interests.
- As a traveler, I want to filter destinations by budget so that I can only see places I can afford to visit.
- As a traveler, I want to view an estimated total trip cost for a 2-day visit so that I can decide if a destination is within my budget before planning.
- As a traveler, I want to see a live weather badge on the destination page so that I can avoid traveling during bad weather conditions.
- As a traveler, I want to view the destination marked on Google Maps so that I can clearly understand where it is located.
- As a traveler, I want to get a route from my current location to the destination so that I can navigate there easily.

Researcher / Student User Stories:

- As a researcher, I want to read detailed historical and cultural information about heritage sites so that I can use accurate data for my academic work.
- As a student, I want to browse destinations by division so that I can study the geographical distribution of tourist spots across Bangladesh.

Administrator User Stories:

- As an admin, I want to log in to a secure dashboard so that only authorized users can manage destination data.
 - As an admin, I want to update bus fare and hotel rates so that travelers always receive accurate and current budget estimates.
 - As an admin, I want to add new tourist destinations to the database so that the platform stays up to date with newly discovered or developed spots.
 - As an admin, I want to manage and delete outdated destination records so that users are not misled by irrelevant information.
-

6. Conclusion

The diagram, for Ghuri-Phiri shows us who is involved. We have Traveler, Researcher/Student and Administrator. These are the people who will use Ghuri-Phiri. There are also two systems that Ghuri-Phiri will work with. These are OpenWeatherMap API and Google Maps API.

We found fourteen things that Ghuri-Phiri can do. These things are connected in ways. The diagram shows what Ghuri-Phiri is supposed to do. It does what we said it should do in the SRS. We made sure that all the things Ghuri-Phiri needs to do are included. This is shown by the people and systems working together with Ghuri-Phiri. We have five things that Ghuri-Phiri needs to do. These are called FR1, FR2, FR3, FR4 and FR5. The diagram shows that Ghuri-Phiri can do all of these things.

Lab Report: Software Development Life Cycle (SDLC) Selection

Course: Software Development Project

Lab: SDLC Model Selection

Project: Ghuri-Phiri – A Smart Tourist Guide & Budget Planner

Group: 5 Members:

Punam (23524202064)

Rifat (23524202096)

Mehrab (23524202112)

Orce (23524202136)

1. Objectives

- To understand different SDLC models.
 - To learn how to select a suitable SDLC model based on project requirements.
 - To analyze factors affecting SDLC selection.
 - To justify the chosen SDLC model for the Ghuri-Phiri project.
-

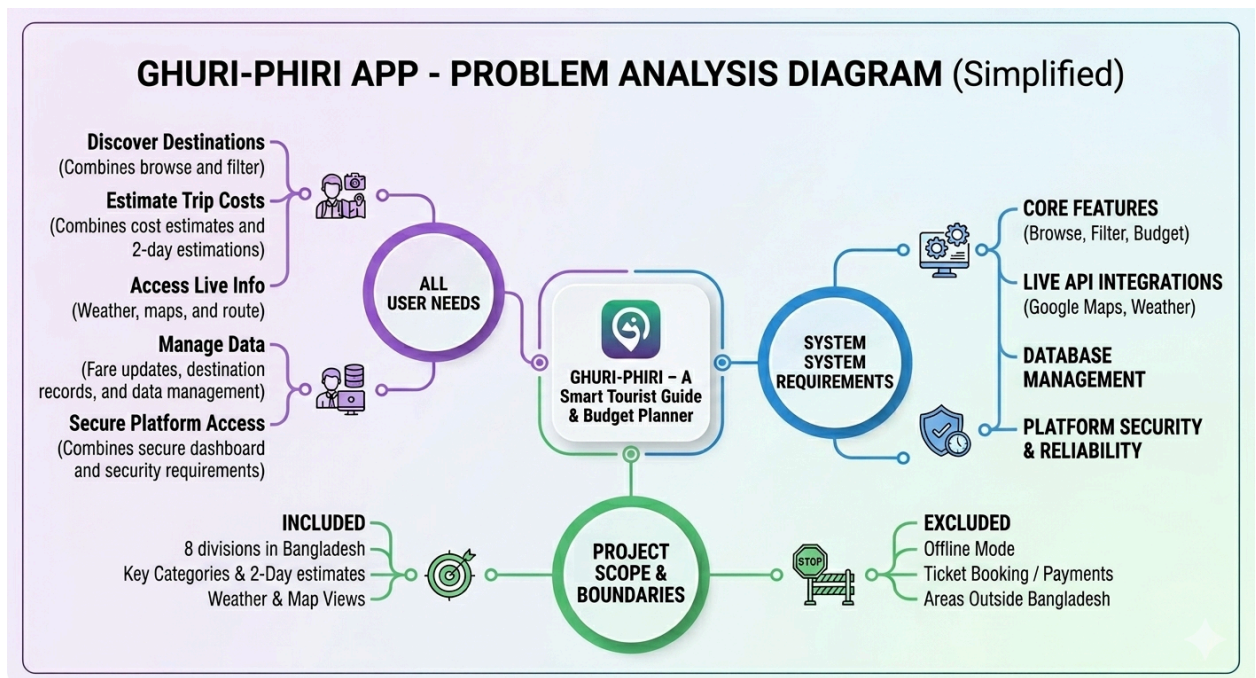
2. Problem Analysis

- Ghuri-Phiri is a web-based tourist guide and budget planner and it is only for Bangladesh.
- Here are the things that the system needs to do:
 - Collect data on all the tourist spots in Bangladesh 8 divisions.
 - Calculate travel costs, like transport, hotel and food costs in time.
 - Use services, like Google Maps and OpenWeatherMap to get more information.
 - Have a dashboard for administrators to update data in real-time.
- The system needs to be working by June 5.

- The Ghuri-Phiri system will help tourists plan their trips and keep track of their expenses.
- It will have all the information that tourists need to plan their trips to Bangladesh.
- The system will use Google Maps to help tourists navigate and OpenWeatherMap to provide weather updates.
- The Ghuri-Phiri system will make it easy for tourists to plan their trips.

Key project characteristics identified:

- Small team of 4 people with jobs.
- The requirements are clearly written down in the SRS document.
- There are 5 modules that work on their own.
- There is some risk because we depend on an API and the data might get old.
- Each of the 5 parts can be built and shown separately.
- This makes it easy to deliver one feature at a time.
- The functional parts are called FR1 to FR5.
- We can work on each one of them independently.
- The team will focus on one module at a time.
- The SRS has all the details we need.



3. SDLC Models Overview

- **Waterfall Model** is a step-by-step approach. You complete each phase before moving to the one. You cannot go back to phases once they're done. This model works well for projects where you know exactly what you need and don't expect any changes.
 - **Iterative Model** is about building in cycles. Each cycle makes your system a little better. It's good when you mostly know what you want but need to make some changes along the way. You build a system first then make it better.
 - **Incremental Model** means you plan your system at the start but build it bit by bit. Each new part adds a feature to what you already have. This works when your needs are clear and can be split into tasks. Your team works on one feature at a time.
 - **Agile Model** is flexible. It relies on getting feedback from stakeholders and delivering work in short bursts. Requirements can change as you go. This approach needs team meetings which can be extra work for small teams.
 - **Prototype Model** involves building a working example on. This helps you get feedback from users before you finish the project. It's useful when you're not sure what you need or when you want to check your design, with users.
-

4. Methodology

Step 1: Analyze the Problem

Ghuri-Phiri has a plan that is written down. This plan has 5 parts that work on their own. Each part is a feature of Ghuri-Phiri. These features are the Global Filter, the Budget Calculator, the Weather Widget, the Map Integration and the Admin Dashboard. The team working on Ghuri-Phiri has people doing different jobs. There is a Database person, a Backend person, a Frontend person and a Tester. Because of this the team will work on one feature of Ghuri-Phiri at a time. The work needs to be done by June 5. By this date each feature of Ghuri-Phiri should be working on its own.

Step 2: Evaluate Project Characteristics

Factor	Assessment
Project Size	Small–Medium
Requirement Clarity	High — SRS already finalized with 5 independent FRs

Risk Level	Moderate (external API dependency)
Time Constraint	High (fixed June 5 interface deadline)
Budget	Low (academic project, free-tier APIs)
Customer Involvement	Low (academic evaluators)
Team Structure	Role-separated (DB, Backend, Frontend, Tester)
Documentation Need	High (lab evaluation requires documented increments)

Step 3: SDLC Model Comparison Matrix

Priority weight: 1 = Low, 2 = Medium, 3 = High. Model score: 1 = Poor fit, 2 = Moderate fit, 3 = Good fit. Weighted score = Priority × Model Score.

Step 4: Select Suitable Model

Selected Model: Incremental (Highest score: 61)

Step 5: Justify Selection

The Incremental model achieved the highest weighted score of 61. The justification is as follows:

Independent Modular Features: Ghuri-Phiri's 5 functional requirements are self-contained and non-overlapping. FR1 (Filter) does not depend on FR3 (Weather Widget) — each can be built, tested, and demonstrated independently. This is the defining characteristic of Incremental development.

Role-Separated Team Structure: Punam (Database), Mehrab (Backend), Orce (Frontend), and Rifat (Testing) each own a layer of the system. In each increment, all four roles contribute to completing one full feature end-to-end — exactly how Incremental development operates.

Fixed Deadline with Measurable Progress: Each increment delivers a working, demonstrable feature before June 5. If time runs short, the team has already delivered FR1–FR3 as fully working increments rather than an unfinished whole system.

5. Proposed Increment Plan

Increment	Duration	Feature	FR Covered	Deliverable
-----------	----------	---------	------------	-------------

Increment 1	Week 1–2	Core UI + Division Listing + Global Filter	FR1	Fully working filtered browse interface
Increment 2	Week 3	Budget Calculator	FR2	Place detail page with dynamic cost estimation
Increment 3	Week 4	Live Weather Widget	FR3	Weather badge integrated on destination page
Increment 4	Week 5	Map Integration	FR4	Google Maps embedded with location routing
Increment 5	Week 6	Admin Dashboard	FR5	Secure admin panel with rate update capability

Final Interface Delivery: June 5

Step 3: SDLC Model Comparison Matrix

Priority weight: 1 = Low, 2 = Medium, 3 = High. Model score: 1 = Poor fit, 2 = Moderate fit, 3 = Good fit. Weighted score = Priority × Model Score.

Criterion	Priority	Waterfall	Iterative	Incremental	Agile	Prototype
Fixed deadline compatibility	3 ▾	3 ▾	2 ▾	3 ▾	2 ▾	2 ▾
Well-documented fixed SRS	3 ▾	3 ▾	3 ▾	3 ▾	1 ▾	1 ▾
Independent modular features (FR1–FR5)	3 ▾	1 ▾	2 ▾	3 ▾	2 ▾	1 ▾

Role-separated team delivery	3 ▾	2 ▾	2 ▾	3 ▾	2 ▾	2 ▾
Handles API-dependent adjustments	2 ▾	1 ▾	2 ▾	3 ▾	3 ▾	2 ▾
Low process overhead	3 ▾	3 ▾	3 ▾	3 ▾	1 ▾	2 ▾
Structured documentation for evaluation	2 ▾	2 ▾	2 ▾	3 ▾	2 ▾	1 ▾
Early working feature delivery	2 ▾	1 ▾	2 ▾	3 ▾	3 ▾	2 ▾
Risk management	2 ▾	1 ▾	2 ▾	2 ▾	2 ▾	1 ▾

Weighted Total Scores:

Model	Calculation	Total
Waterfall	$(3 \times 3) + (3 \times 3) + (3 \times 1) + (3 \times 2) + (2 \times 1) + (3 \times 3) + (2 \times 2) + (2 \times 1) + (2 \times 1)$	40 ▾
Iterative	$(3 \times 2) + (3 \times 3) + (3 \times 2) + (3 \times 2) + (2 \times 2) + (3 \times 3) + (2 \times 2) + (2 \times 2) + (2 \times 2)$	47 ▾
Incremental	$(3 \times 3) + (3 \times 3) + (3 \times 3) + (3 \times 3) + (2 \times 3) + (3 \times 3) + (2 \times 3) + (2 \times 3) + (2 \times 2)$	61 ▾
Agile	$(3 \times 2) + (3 \times 1) + (3 \times 2) + (3 \times 2) + (2 \times 2)$	40 ▾

	$\times 3)+(3 \times 1)+(2 \times 2)+(2 \times 3)+(2 \times 2)$)	
Prototype	$(3 \times 2)+(3 \times 1)+(3 \times 1)+(3 \times 2)+(2 \times 2)+(3 \times 2)+(2 \times 1)+(2 \times 2)+(2 \times 1)$)	33 -

5. Proposed Increment Plan

Increment	Duration	Feature	FR Covered	Deliverable
Increment 1	Week 1–2	Core UI + Division Listing + Global Filter	FR1	Fully working filtered browse interface
Increment 2	Week 3	Budget Calculator	FR2	Place detail page with dynamic cost estimation
Increment 3	Week 4	Live Weather Widget	FR3	Weather badge integrated on destination page
Increment 4	Week 5	Map Integration	FR4	Google Maps embedded with location routing
Increment 5	Week 6	Admin Dashboard	FR5	Secure admin panel with rate update capability

Final Interface Delivery: June 5

Aligned with Finalized SRS: The SRS is already complete and requirements are fixed. Incremental development does not require requirements to evolve — it simply delivers them one at a time in priority order.

Handles API Risk Incrementally: FR3 (Weather Widget) and FR4 (Map Integration) carry API dependency risk. By addressing them in dedicated increments, the team can isolate and resolve API issues without disrupting already-completed increments.

Low Process Overhead: Unlike Agile, Incremental requires no sprint ceremonies, scrum masters, or continuous stakeholder reviews. The team focuses on building and testing each feature in sequence — suitable for an academic team under time pressure.

Iterative (score: 47) is a valid alternative but assumes the team builds a rough version of the entire system first and then refines it cycle by cycle. Given that Ghuri-Phiri's features are independent and the team is role-separated, building a rough full system in the first cycle is not realistic. Incremental better reflects how the project will actually be executed.

Agile (score: 40) performs well on flexibility but loses points because the SRS is already finalized, there is no active end-user giving sprint feedback, and its process overhead is unnecessary for a 4-person academic team.

6. User Stories

- As a traveler I want to be able to filter tourist spots by category like Hill or Beach across all 8 divisions. This way I can find places that I really want to visit based on what I like not where they are.
 - As a budget- traveler I want to see how much a 2-day trip will cost me so I can decide if I can afford to go to a certain place.
 - As a traveler who likes to plan I want to see what the weather is like at my destination so I can avoid going when there are storms or a lot of rain.
 - As a traveler I want to be able to click on a map and have my destination show up on Google Maps so it is easy for me to get there.
 - As an admin I want to be able to update the cost of bus fares and hotel rates in a dashboard so users always get the right information, about how much things cost.
-

7. Conclusion

The **Incremental model** is the way to go for the Ghuri-Phiri project. This is because the Incremental model delivers things one feature at a time which's perfect for the Ghuri-Phiri project. The Ghuri-Phiri project has 5 parts that need to work on their own and the team is

split into different roles. The Ghuri-Phiri project also has a fixed set of requirements and a deadline of June 5.

The Incremental model is good for the Ghuri-Phiri project because it shows that something is working at every step. This makes it easy to see how things are going. It helps to keep the technical parts simple. It also helps to keep the paperwork clean and easy to look at which is important for the people who need to evaluate the Ghuri-Phiri project. The Incremental model is a choice, for the Ghuri-Phiri project.